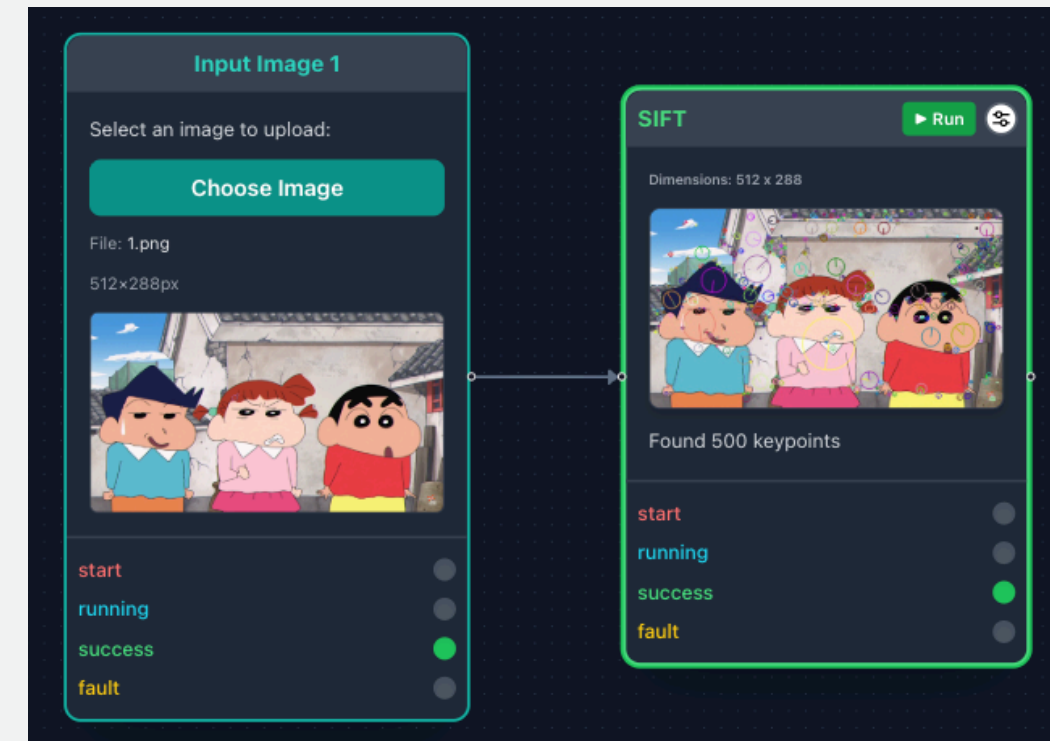


VISUAL IMAGE PROCESSING & EVALUATION RESOURCE (VIPER)

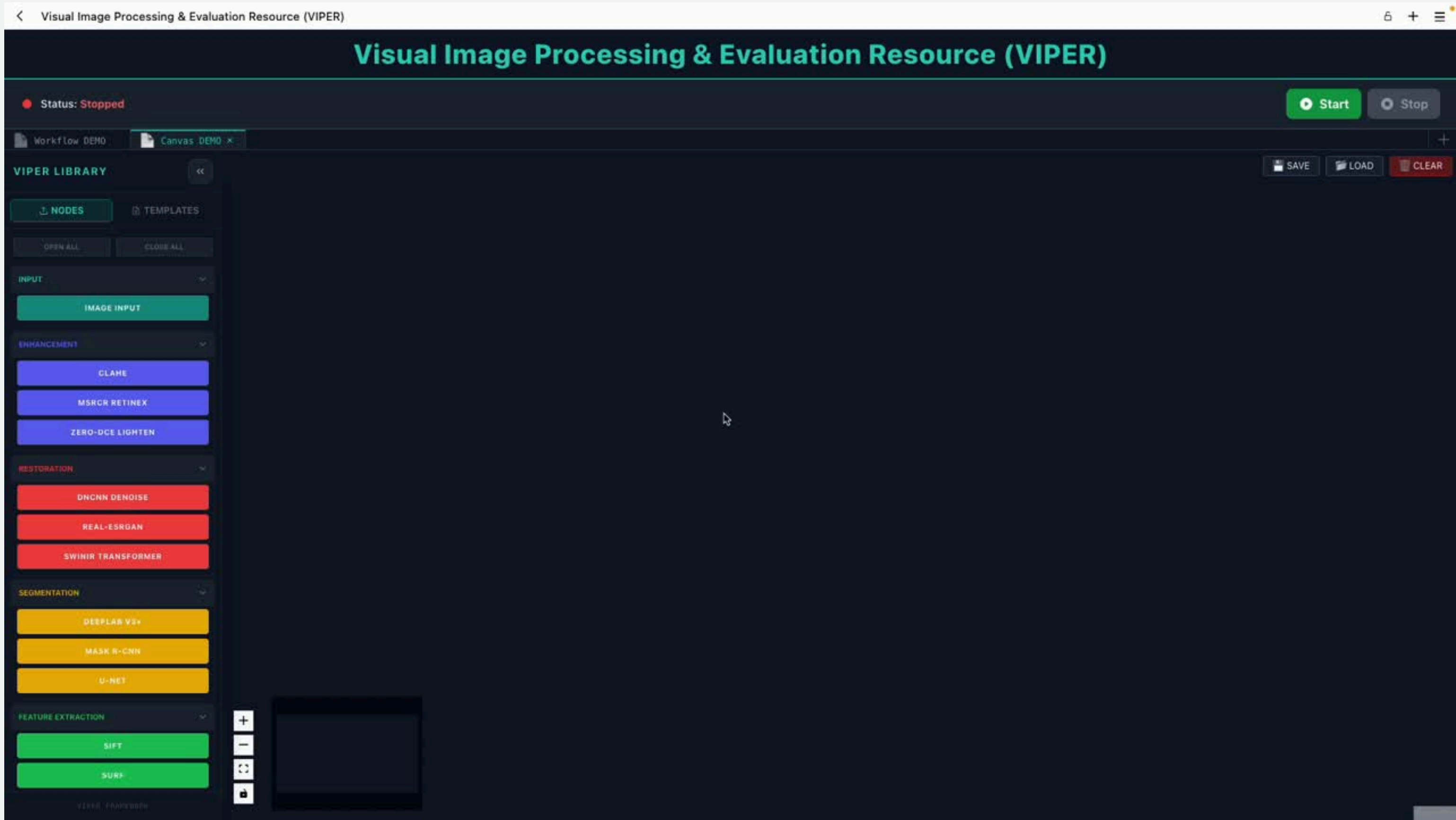
Presentation by
Tosapat Sanpan 6510301002



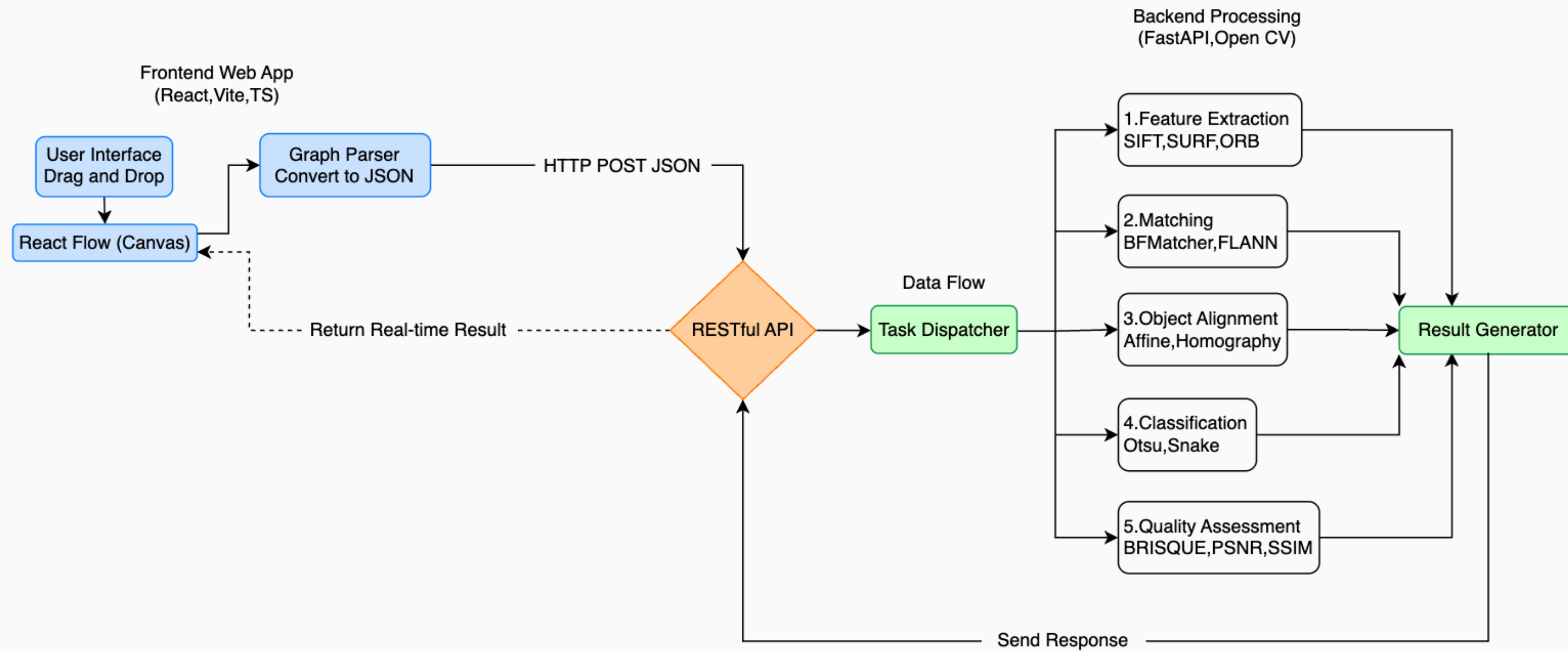
PROBLEM STATEMENT & MOTIVATION



SYSTEM DEMONSTRATION



SYSTEM ARCHITECTURE



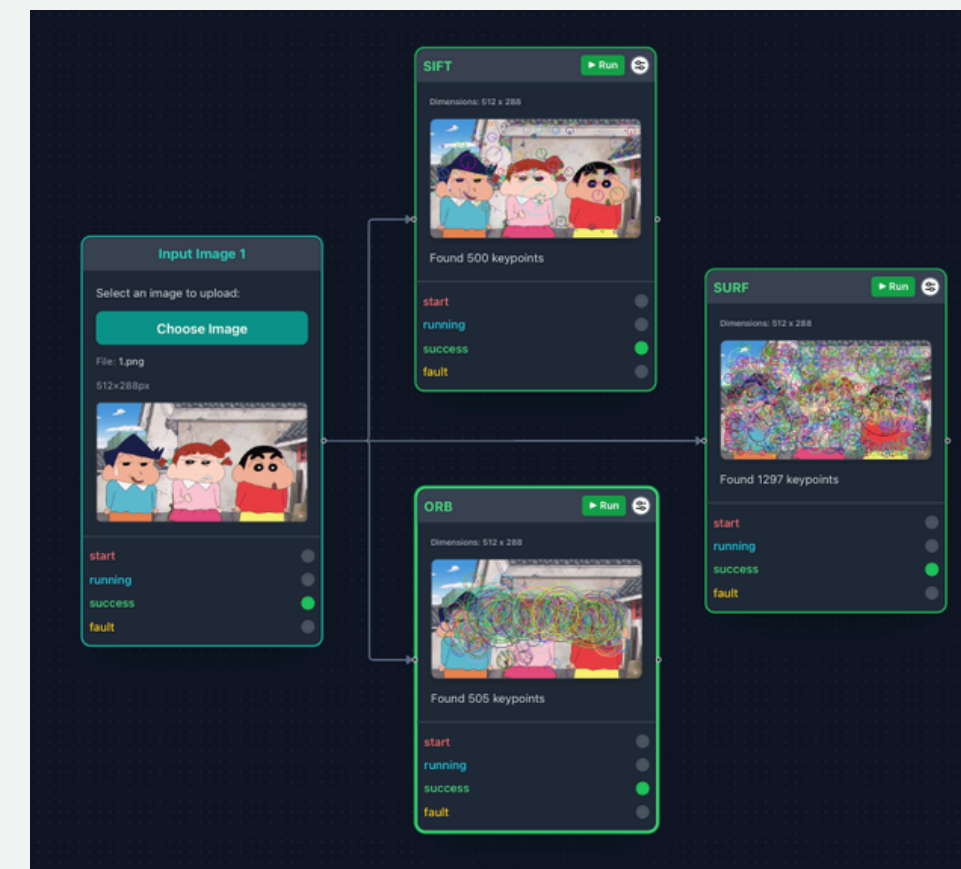
FEATURE EXTRACTION

คำอธิบายและเป้าหมาย (Description & Importance)

- คืออะไร: กระบวนการค้นหา "จุดเด่นที่เป็นเอกลักษณ์" (Keypoints) ของภาพ เช่น บริเวณมุม (Corners), ขอบ (Edges) หรือลวดลายเฉพาะตัว
- เป้าหมายและความสำคัญ: เพื่อสร้าง "Feature" ของภาพ ไว้ใช้เป็นข้อมูลอ้างอิงที่ทนทานต่อการเปลี่ยนแปลง (เช่น ภาพถูกหมุน ย่อ หรือ ขยาย) ซึ่งเป็นรากฐานสำคัญสำหรับการวิเคราะห์ภาพในขั้นต่อไป

อัลกอริทึมที่รองรับ (Supported Algorithms)

- SIFT (Scale-Invariant Feature Transform)
- SURF (Speeded-Up Robust Features)
- ORB (Oriented FAST and Rotated BRIEF)



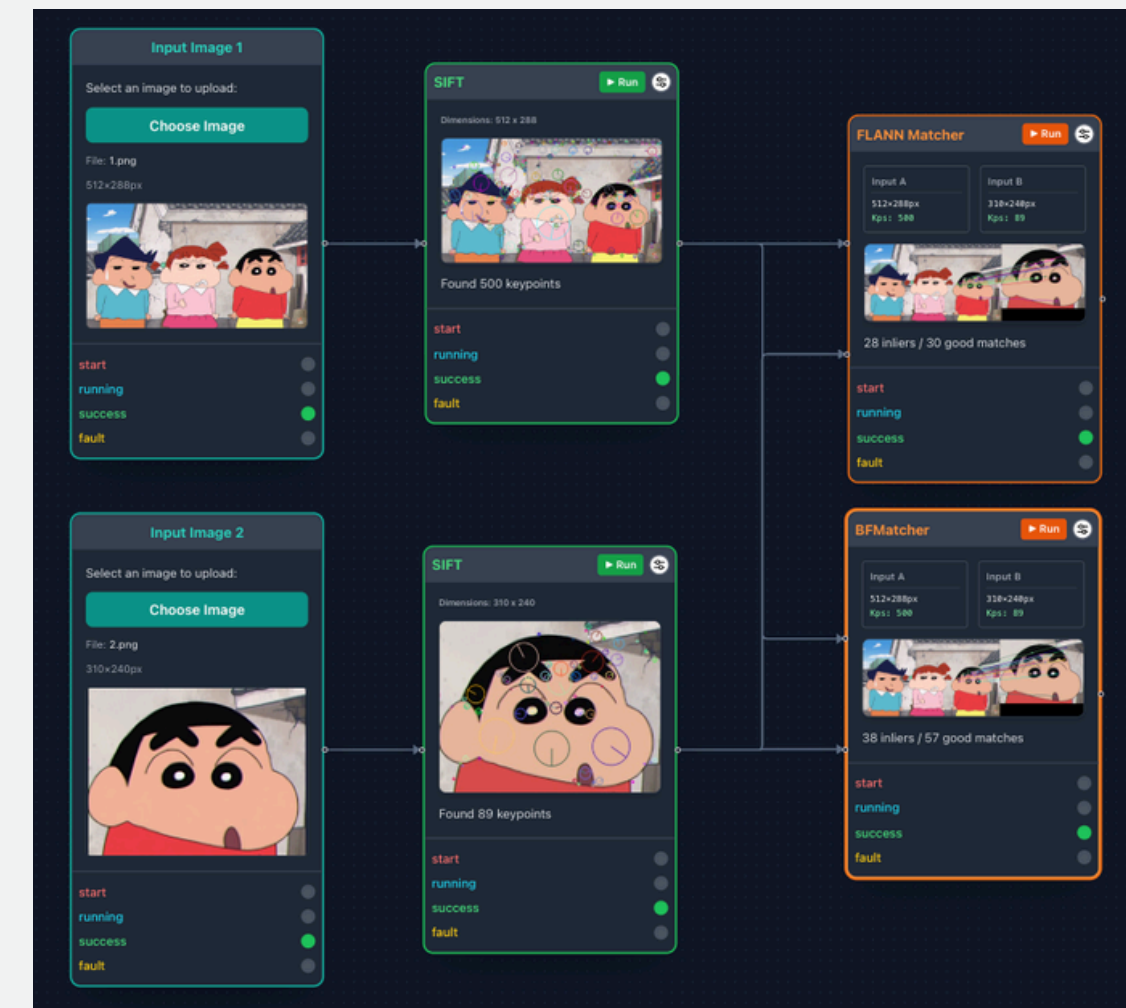
FEATURE MATCHING

คำอธิบายและเป้าหมาย (Description & Importance)

- คืออะไร: กระบวนการนำข้อมูลจุดเด่น (Keypoints) ที่ได้จากขั้นตอนแรกของภาพตั้งแต่ 2 ภาพขึ้นไป มาคำนวณเปรียบเทียบเพื่อหาความเชื่อมโยงและจับคู่จุดที่ตรงกัน
- เป้าหมายและความสำคัญ: เพื่อหาระดับความคล้ายคลึง (Similarity) ระหว่างภาพ ซึ่งเป็นหัวใจสำคัญของงาน Computer Vision ขั้นสูง เช่น การต่อภาพพาโนรามา (Image Stitching) หรือการตรวจจับและค้นหาวัตถุในภาพ (Object Detection)

อัลกอริทึมที่รองรับ (Supported Algorithms)

- BFMatcher (Brute-Force Matcher): เน้นการเปรียบเทียบแบบตรงไปตรงมาเพื่อความแม่นยำสูงสุด
- FLANN (Fast Library for Approximate Nearest Neighbors): เน้นความเร็ว เหมาะสำหรับชุดข้อมูลที่มีขนาดใหญ่



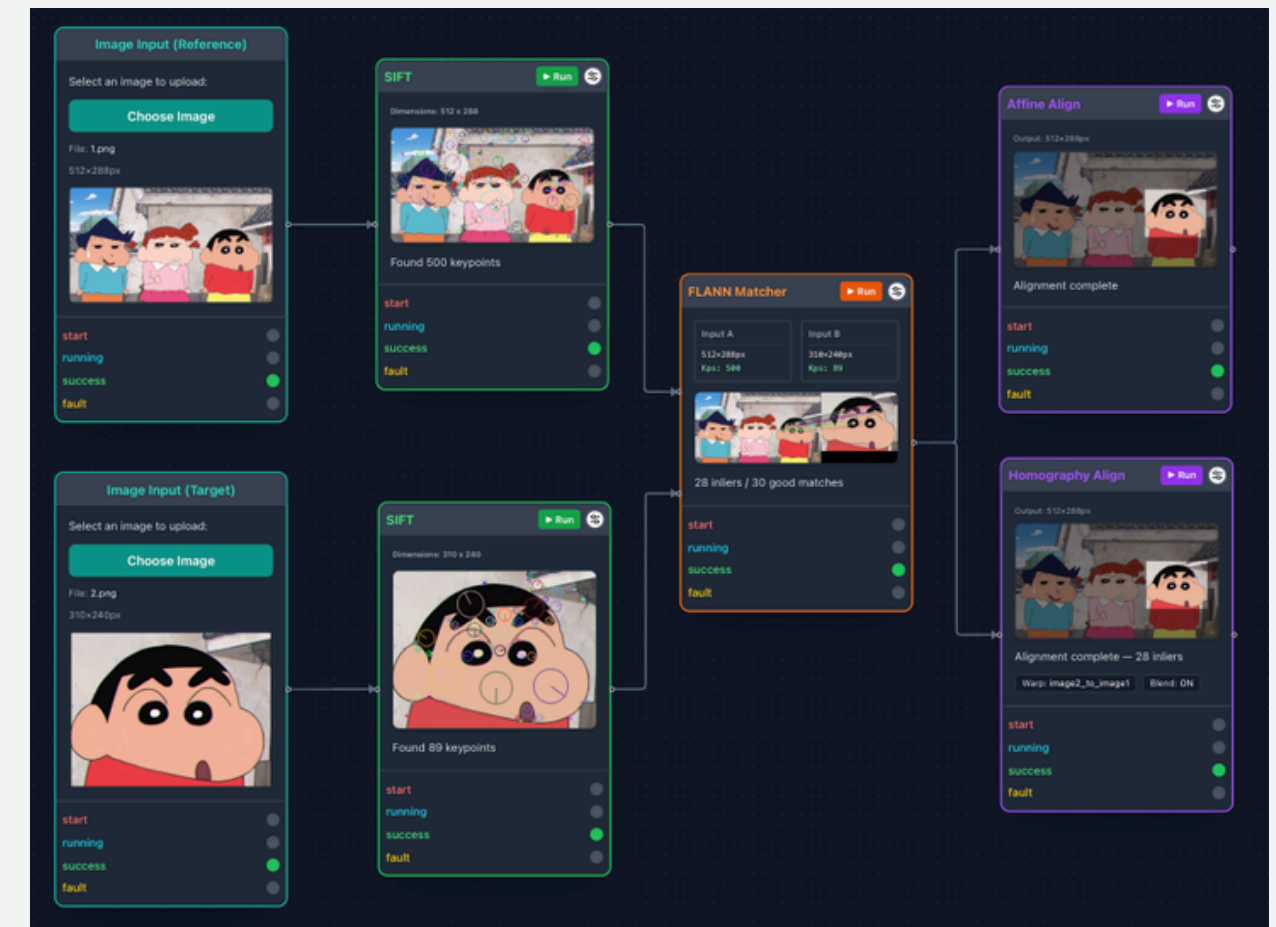
OBJECT ALIGNMENT

คำอธิบายและเป้าหมาย (Description & Importance)

- คืออะไร: กระบวนการดัดแปลงรูปทรงเรขาคณิตของภาพ (Geometric Transformation) โดยใช้ข้อมูลพิกัดจุดอ้างอิงที่ได้จากการจับคู่ (Matching) ในขั้นตอนก่อนหน้า
- เป้าหมายและความสำคัญ: เพื่อปรับแก้ มุมมอง (Perspective), องศา หรือขนาดของภาพที่ถ่ายมาต่างมุมกัน ให้เข้ามาอยู่ในระนาบเดียวกัน ทำให้สามารถนำภาพมาต่อกัน (Stitching) หรือซ้อนทับกันเพื่อเปรียบเทียบได้อย่างสมบูรณ์แบบ

อัลกอริทึมที่รองรับ (Supported Algorithms)

- Homography: การแปลงพิกัดเพื่อดัดมุมมองภาพแบบ Perspective (เช่น ดัดภาพป้ายที่ถ่ายเอียงๆ ให้กลับมาตั้งตรง)
- Affine Transformation: การแปลงรูปทรงเรขาคณิตพื้นฐาน เช่น การเลื่อนตำแหน่ง, การหมุน และการย่อ/ขยาย โดยยังคงรักษาเส้นขนานของภาพไว้



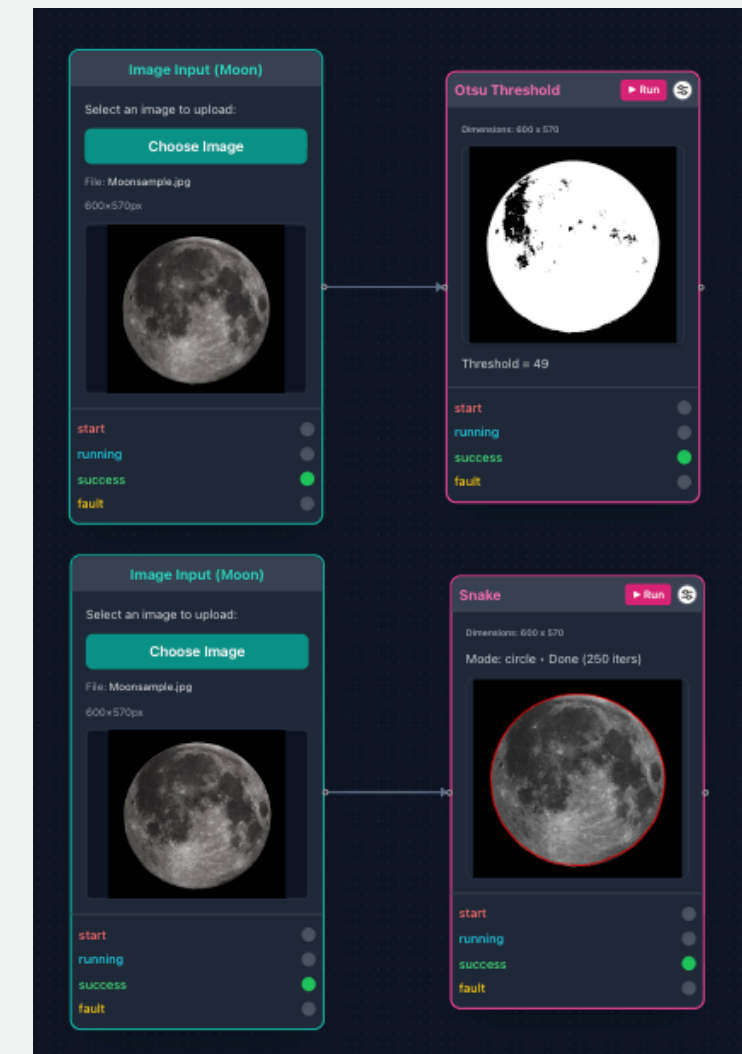
CLASSIFICATION

คำอธิบายและเป้าหมาย (Description & Importance)

- คืออะไร: กระบวนการแยกวัตถุเป้าหมายออกจากพื้นหลัง (Background Subtraction) และการค้นหาขอบเขตของวัตถุ (Contour Detection) โดยอาศัยการวิเคราะห์ค่าความสว่างของพิกเซล
- เป้าหมายและความสำคัญ: เพื่อดึงเอาเฉพาะ "รูปร่าง (Shape)" หรือ "เส้นขอบ (Boundary)" ของวัตถุที่สนใจออกมาให้เด่นชัดที่สุด และตัดข้อมูลพื้นหลังที่ไม่จำเป็นทิ้งไป ซึ่งเป็นขั้นตอนสำคัญในการตีกรอบให้คอมพิวเตอร์รู้ว่าวัตถุที่เราสนใจอยู่ตรงไหนของภาพ

อัลกอริทึมที่รองรับ (Supported Algorithms)

- Otsu's Thresholding: อัลกอริทึมที่ช่วยคำนวณหาค่าจุดตัดความสว่าง (Threshold) ที่เหมาะสมที่สุดแบบอัตโนมัติ เพื่อแยกภาพออกเป็นส่วนที่เป็นวัตถุและพื้นหลัง (แปลงเป็นภาพขาว-ดำ)
- Active Contour (Snake): อัลกอริทึมค้นหาเส้นขอบวัตถุแบบยืดหยุ่น โดยระบบจะสร้างเส้นสมมติที่สามารถหดตัวเข้าหาขอบที่แท้จริงของวัตถุเป้าหมายได้



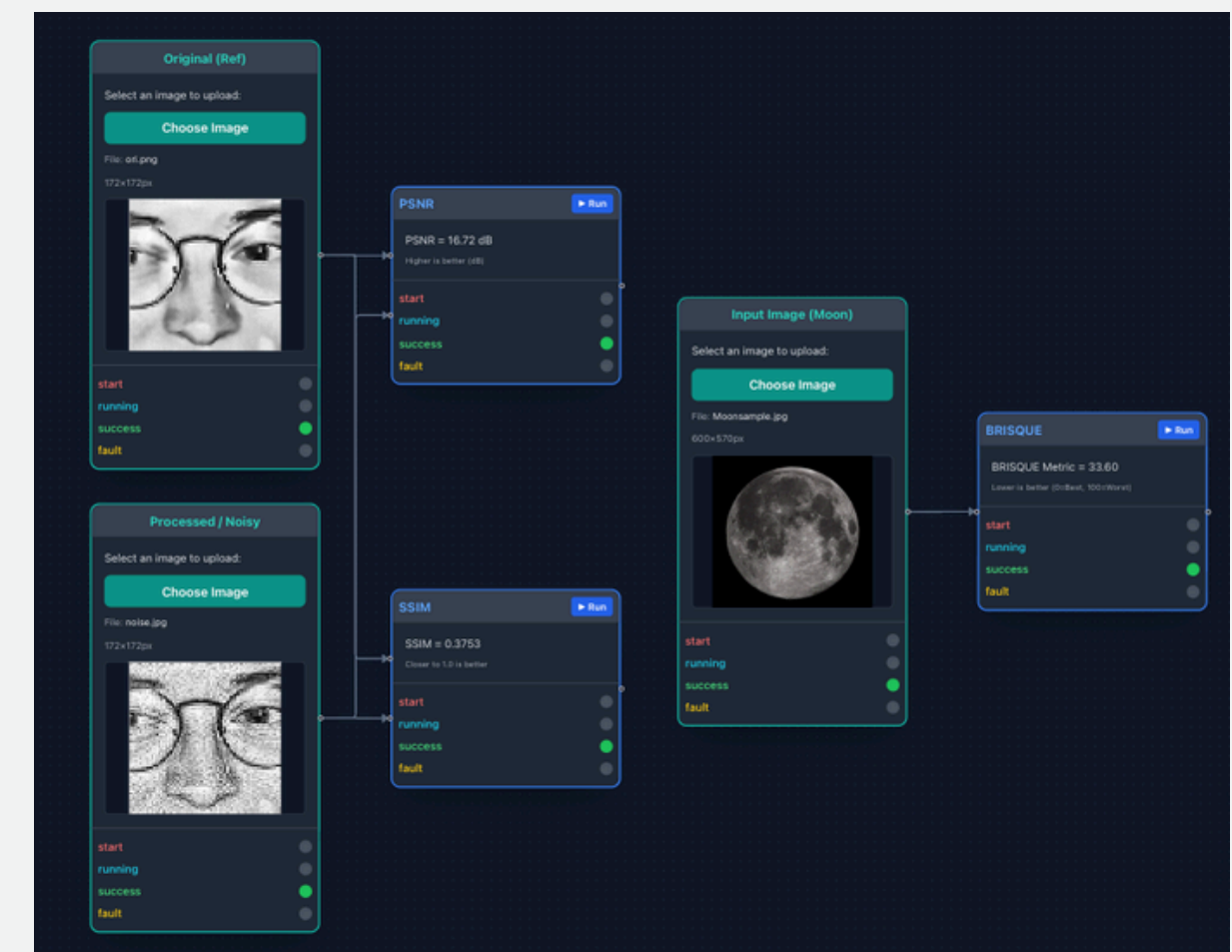
QUALITY ASSESSMENT

คำอธิบายและเป้าหมาย (Description & Importance)

- คืออะไร: กระบวนการทางคณิตศาสตร์และสถิติเพื่อวัดระดับคุณภาพ ความคมชัด หรืออัตราการผิดเพี้ยนของภาพที่ผ่านการประมวลผล
- เป้าหมายและความสำคัญ: เพื่อประเมินประสิทธิภาพของอัลกอริทึมต่างๆ ออกมาเป็น "ตัวเลขที่จับต้องได้" แทนการใช้เพียงสายตาตามนุษย์คาดเดา ซึ่งจำเป็นอย่างยิ่งในการเปรียบเทียบว่าภาพที่ได้มีคุณภาพดีขึ้นหรือแย่ลงเพียงใด

อัลกอริทึมที่รองรับ (Supported Algorithms)

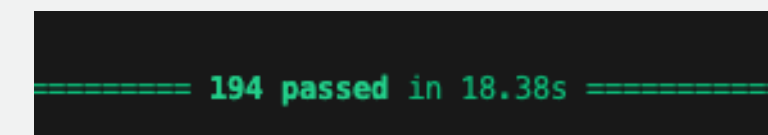
- PSNR & SSIM: การประเมินแบบอ้างอิงภาพต้นฉบับ (Full-Reference) เพื่อวัดค่าความผิดเพี้ยนของพิกเซล และความคล้ายคลึงของโครงสร้างภาพ
- BRISQUE: การประเมินแบบไม่อ้างอิงภาพต้นฉบับ (No-Reference) โดยใช้โมเดลทางสถิติมาวิเคราะห์ความ "เป็นธรรมชาติ" ของภาพ



SYSTEM TESTING & VALIDATION

1. Automated Unit Testing (Backend)

- ใช้เฟรมเวิร์ก unittest ของ Python ทดสอบการทำงานของ Core Algorithms
- ทดสอบกลุ่มถึงกรณี Edge Cases (เช่น ภาพพัง, ภาพ Grayscale, หรือการเรียกใช้อัลกอริทึมผิดประเภท)



```
===== 194 passed in 18.38s =====
```

2. Manual & Integration Testing (Frontend & API)

- ทดสอบการทำงานร่วมกันแบบ End-to-End
- ทดสอบความเสถียรของ UI (การจัดการ Node/Edge) และความสามารถของ API ในการตรวจจับ Error จากพฤติกรรมการใช้งานที่ไม่ถูกต้อง

3. Advisor Validation

- สถาปัตยกรรมและผลลัพธ์ผ่านการตรวจสอบความถูกต้องจากอาจารย์ที่ปรึกษาโครงงาน

CONCLUSION & FUTURE WORK

Project Success

- VIPER Platform: พัฒนา Web Application สำหรับ Image Processing แบบ Node-based สำเร็จตามเป้าหมาย 100%
- Seamless Integration: บูรณาการหน้าบ้าน (React Flow) และหลังบ้าน (OpenCV) ทำงานร่วมกันผ่าน RESTful API ได้อย่างเสถียรและแม่นยำ

Sustainability & Future Work

- Frontend Developer Guide: จัดทำคู่มือสำหรับนักพัฒนาไว้อย่างละเอียด ครอบคลุมตั้งแต่ Tech Stack, Project Structure ไปจนถึง Core Concepts ของระบบ
- "How to Add a New Node": มี Guideline แบบ Step-by-step สำหรับผู้ที่ต้องการเพิ่มโหนดอัลกอริทึมใหม่ๆ หรือปรับแก้ API Integration ในอนาคต
- Future Expansion: โครงสร้างระบบพร้อมรองรับการขยายสเกลขั้นสูง algorithm ต่างๆเข้ามาเชื่อมต่อในเฟสถัดไป

Frontend Developer Guide: n2n-image-processing

Project Name: n2n-image-processing (Frontend)

Framework: React + Vite + React Flow

Introduction & Overview

1.1 โปรเจกต์คืออะไร? (What is this?)

n2n-image-processing คือ Web Application ที่ช่วยให้ผู้ใช้งานสามารถสร้างและทดสอบกระบวนการประมวลผลภาพ (Image Processing Pipelines) ได้ผ่านหน้าจอแบบกราฟิก (GUI) โดยใช้หลักการ

Node-based Interface

ผู้ใช้งานสามารถ:

- ลากและวางกล่องเครื่องมือ (Nodes) เช่น การโหลดภาพ, การปรับแต่งภาพ, หรือการตรวจจับวัตถุ
- เชื่อมต่อเส้น (Edges) เพื่อกำหนดลำดับการทำงาน
- กดปุ่ม "Run" เพื่อดูผลลัพธ์การประมวลผลได้ทันทีแบบ Real-time โดยระบบจะส่งคำสั่งไปทำงานที่ Python Backend

1.2 สถาปัตยกรรมระบบ (System Architecture)

ส่วน Frontend ทำหน้าที่เป็น "Visual Orchestrator" หรือตัวควบคุมการทำงาน โดยมีการติดต่อสื่อสารกับระบบดังนี้:

- User Interface: สร้างด้วย React และ React Flow เพื่อจัดการกราฟ
- API Communication: ใช้ Axios ส่งข้อมูล JSON (Parameters) และ File ไปยัง Backend (FastAPI)
- Result Rendering: รับ URL ของรูปภาพผลลัพธ์จาก Backend มาแสดงผลบน Node



THANK YOU

